buildandteach

PYSPARK QUERY · PART 1 QUICK REFERENCE

Select · filter · withColumn · nulls

Shape rows before you aggregate. Count nulls before you clean.
Free Edition: serverless compute + Unity Catalog. This part is read-only.

1. Inputs + derived total

```
# 8 fictional orders; qty/price/customer nullable on purpose
orders = orders.withColumn("order_total",
  (F.col("quantity")*F.col("unit_price")).cast("decimal(12,2)"))
# Actual totals 101-108: 20,25,25.50,30,12,null,null,30
```

INPUT CHECK: 8 orders (2 incomplete qty/price rows).

2. Valid filter + Books NORTH/WEST

```
valid = orders.filter(F.col("quantity").isNotNull() &
                      F.col("unit_price").isNotNull())
books = (valid.select("order_id","customer","region","category","order_total")
  .where((F.col("category")=="Books") & F.col("region").isin("NORTH","WEST"))
  .orderBy("order_id"))
```

Actual: 101=20.00, 103=25.50 · sum 45.50 · 6 valid overall.

3. High-value ordered list

```
high = (valid.filter(F.col("order_total")>=25)
  .orderBy(F.col("order_total").desc(), F.col("order_id").desc()))
```

Actual: 108,104,103,102 · total 110.50 (secondary sort breaks $30 ties).

Null hygiene + practice

4. Inspect nulls (do not clean yet)

```
orders.printSchema()
orders.select("quantity","unit_price","order_total").summary("count","mean","min","max")
null_counts = orders.select(
  *[F.count(F.when(F.col(c).isNull(),1)).alias(c)
    for c in ("customer","quantity","unit_price")])
```

Actual nulls: customer 1 · quantity 1 · unit_price 1. Summary: qty 7 / price 7 / total 6.

Practice prediction

Toys in SOUTH or NORTH from valid rows, order by total desc → 108,104,102.

Associate §3 foundation (Aug 2026). Partial coverage. No exam guarantee.
Independent education. Fictional-data project. Part 1 of 3.